8 Roteiro Prático 07 — Manipulação da Árvore DOM

Objetivos

- Dominar os métodos de seleção: `getElementById` e `querySelector`;
- Manipular dinamicamente o conteúdo da página (`innerText`, `innerHTML`);
- Alterar estilos CSS e gerenciar classes (`classList`);
- Criar e remover elementos HTML via JavaScript (`createElement`, `appendChild`).

O que é o DOM?

O **DOM (Document Object Model)** é a representação em árvore dos elementos do seu HTML que o navegador cria para que o JavaScript possa interagir com eles.

Sintaxe Moderna: A Metáfora da Seta (Arrow Functions)

Dicas Úteis

O Atalho do Chef (=>)
Imagine que escrever código é como pedir um prato em um restaurante. Na forma antiga (`function`), você tinha que preencher um formulário longo: "Eu declaro que gostaria de uma função que recebe um pedido e faz tal coisa...".
Com a **Arrow Function** (=>), você usa um atalho. O símbolo => é uma seta que aponta diretamente do **evento** para a **ação**. É como dizer: "Quando o botão for clicado **=>** mude a cor". É mais rápido, limpo e moderno!

Parte 1 — Selecionando e Alterando Elementos

A Intenção do Código

Nesta primeira atividade, vamos ensinar o navegador a "olhar" para um título específico e trocar a roupa (cor) dele. O JavaScript vai localizar o ID único e aplicar o novo estilo instantaneamente.

Atividade prática 1 — Seletores e Texto

Crie um arquivo `dom-test.html` e implemente:

```
<h1 id="titulo">Olá Mundo!</h1>
<button onclick="mudarTexto()">Clique Aqui</button>

<script>
function mudarTexto() {
    const titulo = document.getElementById("titulo");
    titulo.innerText = "JavaScript funcionando!";
    titulo.style.color = "blue";
}
</script>
```

Parte 2 — Tabelas de Referência de Métodos

Guia de Métodos de Seleção

Método	Descrição
`getElementById()`	Busca um elemento pelo seu ID único.
`querySelector()`	Busca o primeiro elemento que valide o seletor CSS (ex: .classe, #id, tag).
`querySelectorAll()`	Busca uma lista de todos os elementos que validam o seletor.

Parte 3 — Criação Dinâmica de Elementos

Este é um dos conceitos mais importantes: fazer o HTML "aparecer" na tela via código.

A Intenção do Código

Vamos criar do zero novos elementos, como se estivéssemos montando peças de LEGO na tela. Em vez de escrever o HTML manualmente, o JavaScript vai fabricar uma nova `<li>` toda vez que você clicar no botão.

Atividade prática 2 — A Fábrica de Elementos

Vamos criar uma lista onde cada clique no botão adiciona um novo item:

```
<ul id="minhaLista"></ul>
<button id="add">Adicionar Item</button>

<script>
const lista = document.getElementById("minhaLista");
const botao = document.getElementById("add");

botao.addEventListener("click", () => {
    // 1. Criar o elemento
    const novoItem = document.createElement("li");

    // 2. Dar conteúdo ao elemento
    novoItem.innerText = "Novo Item na Lista";

    // 3. Adicionar na página
    lista.appendChild(novoItem);
});
</script>
```

Parte 4 — Praticando com Estados e Atributos

A Intenção do Código

Agora vamos lidar com números e imagens. No primeiro exemplo, o JavaScript vai guardar um valor na memória (uma variável) e aumentá-lo a cada clique. No segundo, vamos manipular o atributo `src` para trocar uma imagem por outra, simulando um interruptor.

Atividade prática 3 — O Contador de Cliques

Crie um contador simples:

```
<h2 id="contador">0</h2>
<button id="btnSoma">Contar +1</button>

<script>
let valor = 0;
const display = document.getElementById("contador");

document.getElementById("btnSoma").addEventListener("click", () => {
    valor++;
    display.innerText = valor;
});
</script>
```

Atividade prática 4 — Luz Acesa

Troque a imagem dinamicamente (use links de imagens de lâmpadas ou placeholders):

```
<img id="lampada" src="off.png" style="width:100px">
<button id="btnInterruptor">Ligar/Desligar</button>

<script>
const img = document.getElementById("lampada");
let acesa = false;

document.getElementById("btnInterruptor").addEventListener("click", () => {
    acesa = !acesa;
    img.src = acesa ? "on.png" : "off.png";
});
</script>
```

Laboratório de Mini-Jogos Lúdicos

A Intenção do Código

Para consolidar o aprendizado, vamos transformar o código em jogo! Nestes desafios, o JavaScript deixa de ser apenas uma ferramenta de formulário e passa a gerenciar estados de jogo, aleatoriedade e reações rápidas ao mouse.

Atividade prática 5 — Jogo de Adivinhação

Conceito: O computador escolhe um número e você tenta adivinhar. O JS compara os valores e te dá dicas.

```
<input type="number" id="palpite" placeholder="0 a 10">
<button id="btnChutar">Chutar!</button>
<p id="feedback"></p>

<script>
const segredo = Math.floor(Math.random() * 11);
const campo = document.getElementById("palpite");
const texto = document.getElementById("feedback");

document.getElementById("btnChutar").addEventListener("click", () => {
    const chute = Number(campo.value);
    if (chute === segredo) {
        texto.innerText = " Parabéns! Você acertou!";
        texto.style.color = "green";
    } else {
        texto.innerText = chute > segredo ? "Muito alto!" : "Muito baixo!";
        texto.style.color = "red";
    }
});
</script>
```

Atividade prática 6 — O Botão Fugitivo

Conceito: Um botão que foge do mouse! Vamos aprender a mudar a posição absoluta do elemento e contar quantas vezes você falhou.

```
<p>Erros: <span id="contadorErros">0</span></p>
<button id="fugitivo" style="position: absolute; left: 50%; top: 50%;">
    Pegue-me!
</button>

<script>
const btn = document.getElementById("fugitivo");
const placar = document.getElementById("contadorErros");
let erros = 0;

btn.addEventListener("mouseover", () => {
    erros++;
    placar.innerText = erros;

    // Mudar posição aleatoriamente
    const x = Math.random() * (window.innerWidth - 100);
    const y = Math.random() * (window.innerHeight - 50);

    btn.style.left = `${x}px`;
    btn.style.top = `${y}px`;
});

btn.addEventListener("click", () => alert(" Você me pegou!"));
</script>
```

Atividade prática 7 — Jokenpô (Emoji Edition)

Conceito: Pedra, Papel e Tesoura. O JavaScript decide o sorteio do computador e compara com a sua escolha.

```
<button onclick="jogar(’’)"></button>
<button onclick="jogar(’’)"></button>
<button onclick="jogar(’’)"></button>
<h3 id="resultadoJogo">Escolha um!</h3>

<script>
const opcoes = [’’, ’’, ’’];

function jogar(player) {
    const pc = opcoes[Math.floor(Math.random() * 3)];
    const res = document.getElementById("resultadoJogo");

    if (player === pc) {
        res.innerText = ‘Empate! Ambos escolheram ${pc}‘;
    } else if (
        (player === ’’ && pc === ’’) ||
        (player === ’’ && pc === ’’) ||
        (player === ’’ && pc === ’’)
    ) {
        res.innerText = ‘Você Ganhou! ${player} vence ${pc}‘;
    } else {
        res.innerText = ‘Você Perdeu! ${pc} vence ${player}‘;
    }
}
</script>
```

9 Roteiro Prático 08 — Requisições Assíncronas e Fetch API

Objetivos

- Compreender o formato de dados JSON;
- Utilizar a Fetch API para buscar dados externos;
- Integrar o consumo de APIs com a manipulação do DOM.

Entendendo o JSON

JSON é o formato padrão de troca de mensagens na web moderna. Ele funciona como um objeto JavaScript formatado como texto.

A Metáfora da Esteira de Produção (Fetch Chain)

Dicas Úteis

A Esteira de Pedidos

Imagine uma esteira de produção em uma lanchonete:

1. `fetch` [O Pedido] **=>** Você faz o pedido no balcão (internet).
2. `json()` [A Cozinha] **=>** O cozinheiro recebe o pedido bruto e o transforma em um prato pronto para comer (objeto JS).
3. `dados` [O Garçom] **=>** Agora você pode pegar o prato e colocar na mesa (mostrar na tela).

O encadeamento de `.then() => ...` é o movimento da esteira passando o pedido de uma mão para a outra.

Exploração Prática — O Projeto ViaCEP

A Intenção do Código

O fluxo aqui é completo: o usuário digita um CEP **=>** Nós perguntamos aos correios via internet **=>** Eles nos devolvem um pacote (JSON) **=>** Nós abrimos esse pacote e usamos o DOM para mostrar o endereço na tela sem recarregar a página.

Atividade prática 3 — Integração Completa (Fetch + DOM)

Integre a busca de CEP no seu projeto:

```
const cep = document.getElementById("cep").value;

fetch(‘https://viacep.com.br/ws/${cep}/json/‘)
    .then(response => response.json())
    .then(dados => {
        if (dados.erro) {
            document.getElementById("resultado").innerText = "Erro!";
        } else {
            document.getElementById("resultado").innerText =
               ‘${dados.logradouro}, ${dados.localidade}‘;
        }
    });
```

Tabela de Referência — Comandos de Rede

Comando	O que faz?
`fetch(url)`	Inicia a requisição para o servidor.
`.then()`	Aguarda a promessa ser resolvida para executar a ação.
`response.json()`	Converte o pacote de texto recebido em objeto JS.